

BEE 4750 Homework 1: Introduction to Using Julia

Name: Muhammad Islam

ID: 5384772

Due Date

Thursday, 9/11/25, 9:00pm

Overview

Instructions

- Problem 1 consist of a series of code snippets for you to interpret and debug. You will be asked to identify relevant error(s) and fix the code.
- Problem 2 asks you to write code to implement several simple functions using more general programming and some which are unique to Julia (which might mean that you need to look at the documentation or use Julia's help function, which is accessed with `?`, e.g. `?sum` for help with the `sum()` function) or to analyze Julia syntax.
- Problem 3 asks you to convert a verbal description of a wastewater treatment system into a Julia function, and then to use that function to explore the impact of different wastewater allocation strategies.

Load Environment

The following code loads the environment and makes sure all needed packages are installed. This should be at the start of most Julia scripts.

```
In [1]: import Pkg
        Pkg.activate(@__DIR__)
        Pkg.instantiate()
```

Activating project at `~/Desktop/bee4750-hw1-mai58`

Standard Julia practice is to load all needed packages at the top of a file. If you need to load any additional packages in any assignments beyond those which are loaded by default, feel free to add a `using` statement, though [you may need to install the package](#).

```
In [2]: using Random
        using Plots
```

```
using GraphRecipes
using LaTeXStrings
using Distributions
```

```
In [3]: # this sets a random seed, which ensures reproducibility of random number ge
Random.seed!(1)
```

TaskLocalRNG()

Problems (Total: 30 Points)

Problem 1 (12 points)

The following subproblems all involve code snippets that require debugging. You are encouraged to use online resources (e.g. Julia documentation and forums, Stack Overflow, Reddit, etc) to help find diagnose error messages and find solutions, but make sure that you clearly document which resources you used and how you used them.

Problem 1.1

You've been tasked with writing code to identify the minimum value in an array. You cannot use a predefined function. Your colleague suggested the function below, but it does not return the minimum value.

```
In [4]: function minimum(array)
        # initialize the minimum value counter
        # old code: min_value = 0
        min_value = array[1]
        # update minimum values
        for i in 1:length(array)
            if array[i] < min_value
                min_value = array[i]
            end
        end
        # return found minimum
        return min_value
    end

    array_values = [89, 90, 95, 100, 100, 78, 99, 98, 100, 95]
    @show minimum(array_values);
```

minimum(array_values) = 78

What is the logical flaw in the code? Fix it and show that your fixed code solves the problem.

The logical flaw in the original code is that `min_value` was initialized as 0. This is a problem because if all the numbers in the input array are positive, none of them will ever be less than 0 and the loop will never update `min_value`, and the function will incorrectly return 0 instead of the actual minimum of 78.

Problem 1.2

Your team is trying to compute the average grade for your class, but the following code produces an `UndefinedVarError`.

```
In [5]: # enter student grade vector
student_grades = [89, 90, 95, 100, 100, 78, 99, 98, 100, 95]
# compute class average
function class_average(grades)
    average_grade = mean(student_grades)
    return average_grade
end

average_grade = class_average(student_grades)
@show average_grade;
```

```
average_grade = 94.4
```

What does this `UndefinedVarError` mean? Why does this code produce one? Fix the error and solve the problem.

The error `UndefinedVarError` means that the code is attempting to access a variable that has not been defined in the current scope. In the original code, this occurred because the function returned the mean of `student_grades`, but the program then attempted to display `average_grade`, which had never been assigned. Since `average_grade` did not exist, the `UndefinedVarError` came up. I fixed the issue by defining the variable, `average_grade`, to be equal to `class_averages(student_grades)`. Thus, there is a variable equal to the average grade that can be accessed and displayed and the result is 94.4.

Problem 1.3

Your team wants to know the expected payout of an old Italian dice game called *passadieci* (which was analyzed by Galileo as one of the first examples of a rigorous study of probability). The goal of *passadieci* is to get at least an 11 from rolling three fair, six-sided dice. Your strategy is to compute the average wins from 1,000 trials, but the code you've written below produces a `MethodError`.

```
In [6]: # function to simulate a passedieci roll: 3 6-sided dice
function passedieci()
    # this rand() call samples 3 values from the vector [1, 6]
    roll = rand(1:6, 3)
    return roll
end

# set number of trials and initialize outcome vector
n_trials = 1_000
# old code: outcomes = zero(n_trials)
outcomes = zeros(n_trials)
# simulate number of passedieci rolls and count wins
for i = 1:n_trials
```

```

    # old code: outcomes[i] = (sum(passadieci()) > 11)
    outcomes[i] = (sum(passadieci()) >= 11)
end
win_prob = sum(outcomes) / n_trials # compute average number of wins
@show win_prob;

```

win_prob = 0.502

What does this `MethodError` mean? Why does this code produce one? Fix the error and solve the problem.

The `MethodError` happened because the original line `outcomes = zero(n_trials)` did not create a vector but instead returned just the number 0, which cannot be indexed. This caused the program to crash when I tried to assign values with `outcomes[i]`. I fixed the error by changing that line to `outcomes = zeros(n_trials)`, which properly creates a vector of length 1,000 that can hold the results of each trial. I also corrected the condition to `>= 11` so that the code matched the rules of `passadieci`. After making these changes, the program ran successfully and produced an estimated winning probability of about 0.502, which is very close to the exact probability of 0.5.

Problem 1.4

You're interested in writing some code to remove the mean of a vector from all of its components. You've written the following code and tried to test it on a random vector, but your code returns a `MethodError`.

```

In [7]: # function to remove mean from a vector
function remove_mean(vect)
    # fuction to compute the mean
    function compute_mean(vect)
        # old code: element_sum = 0
        element_sum = 0.0 # initialize sum
        # compute mean and return
        for v in vect
            element_sum += v
        end
        return element_sum / length(vect)
    end

    m = compute_mean(vect) # compute mean
    # return demeaned vector
    # old code: return vect - m
    return vect .- m
end

random_vect = rand(1_000)
@show remove_mean(random_vect)

```

```
remove_mean(random_vect) = [0.41919142411294885, -0.04477120169153992, -0.15
987063495003362, -0.3165390583665356, -0.04345183269168251, 0.46004286612673
56, -0.4814808404373695, 0.05265958609672772, -0.019033198532207418, 0.11752
715148103854, -0.2530763598508532, -0.3987461186978969, -0.4051668577883766,
-0.14537708102382318, -0.021518846454485097, 0.2393846697392613, -0.13828803
455335292, 0.28932439329980286, -0.44190436643288566, -0.41462031768210295,
-0.1481768399620188, -0.4467685839176895, 0.16354702958537648, -0.3248934496
17489, -0.22436376824831783, 0.18691945805247367, -0.2843753736563317, 0.308
32500598057266, 0.2757047181327851, 0.44059511305789445, 0.1283259223081242
3, 0.1935791502153814, 0.2896691235161918, -0.05389047441266104, 0.121877293
0619838, -0.24482626621372394, 0.22316484875624798, 0.4597763846856109, -0.3
3758006386107464, 0.19126011359930628, -0.3305504301513418, -0.2083696353735
1572, 0.4435549655840101, 0.14379124696474477, 0.3902589386802747, -0.255140
0209756388, 0.20955546224160848, -0.28102322961489257, 0.35423039564489645,
0.0008391745616401769, 0.1508113023389701, 0.3165514303483038, 0.33381995881
265836, -0.25874845790955914, -0.27673360591837937, -0.0007934120156289914,
0.22668694762632335, -0.18130440850453589, 0.1611220998432945, 0.41438315503
372103, -0.47605581768195804, 0.13759996530853036, 0.012303750851967332, 0.0
5474333583323354, 0.05674873539222247, -0.20931151833870143, 0.0686806490801
9145, -0.32562218174673396, -0.02151382371422117, 0.2668401976299446, -0.185
51349518546956, 0.15041082489831303, -0.0965714889868039, -0.000469229469172
4941, -0.18760687449477065, -0.14070211700337032, 0.06558917891482641, -0.02
7341731592545382, 0.23102463582196553, 0.06805685807907136, -0.1273339967098
417, 0.06872072975944044, -0.4351866842780875, 0.08192316549577516, -0.06823
117543619228, -0.47466565380862036, 0.01536041114638853, -0.3966391714960999
5, -0.49118364199876896, -0.32440283596524067, -0.45929898802686886, 0.48754
6204143507, 0.16199721906446451, -0.11173484564932612, 0.30953201913083817,
0.3533891356278227, -0.01374434448527595, 0.35221050057003134, -0.4080398900
832587, 0.1916781572156967, 0.38491101777523096, -0.06262558149368314, -0.25
84261384093339, -0.02058027700570364, 0.029767247109363182, 0.09829899578795
676, 0.36862104516519467, -0.04049851501202151, -0.02451257831649367, 0.2387
2159296720663, -0.43471283792278137, 0.5005173605455058, -0.0028406276403556
42, 0.30693526044196295, 0.2624502066122705, 0.14879977547274814, -0.2607310
075648748, -0.30432965626447694, -0.4582329741296691, 0.28294253368942757, -
0.42488489628620174, -0.06887295148058803, -0.3578532931633531, 0.0106942161
64155002, -0.1300636669242483, -0.2236053968914793, 0.4937290646726642, 0.10
744108010816622, 0.2397445610538399, 0.18043719455238816, 0.2037880795297688
5, -0.37521080187835876, 0.2149243281642328, -0.4385181244110269, 0.00316465
6620304518, -0.42696425626437073, 0.13044327145388335, 0.48946087271729855,
0.18573311080798738, 0.37757282152557015, -0.44175289502572, 0.1359872365082
3497, -0.46883792244874245, 0.332322041801283, -0.06820823891219419, -0.4482
458514329116, 0.15644094785207496, 0.011389378762457936, -0.265519183339325
2, -0.14098866878346494, 0.43511831257329125, -0.10698502287714917, 0.274755
7727252449, -0.11135067440856572, -0.312571823966751, -0.0964996830418422, -
0.4071223466005701, -0.339909131770448, 0.32124930985414013, -0.178154165738
51128, -0.32299999576821903, -0.4391334373900422, 0.3460687881824539, -0.356
3763644221476, 0.4241763843557216, -0.07106708974369991, -0.4807355211838707
5, -0.36282771033745276, -0.10170514442809553, -0.3874213483458635, -0.06366
313685498448, -0.36189397399212797, 0.1353470697493142, 0.3381849472921302,
0.44426571125932424, 0.16546002855246056, 0.3057087807499439, 0.044018196971
074786, -0.22036385505262646, 0.16068380802066284, -0.17289279939692792, 0.0
7262603345332708, -0.16543520571600723, 0.2615096873183954, -0.3324980006708
0126, -0.33707257597262397, -0.39430435018795407, -0.38569118499164434, 0.44
629627280874307, -0.4869674341251501, -0.3312289940097387, -0.38953877097727
02, 0.30079774149396254, 0.40200292174854335, 0.27589158410778414, -0.165451
46212269213, -0.10479201187956311, 0.2601922460622771, 0.4516850470357491,
```

0.3568116750940933, -0.11919605775674746, 0.17840970256865774, 0.05364206405
0338334, -0.16773186631111925, 0.26902415185692286, -0.10585200893105007, -
0.36465999877299027, 0.36622475673594423, 0.3028206300376588, -0.19888672124
52835, -0.17403474550407128, 0.22869636073398247, -0.3116948427723729, -0.06
287648120820566, 0.4037603361405775, 0.3235983570075992, -0.282328629007275
2, -0.19354019690219715, -0.25708826204365964, 0.27406041970796857, -0.37465
78373784467, 0.18235928886884423, 0.4491181843916726, 0.47452313323031203,
0.04426331190585753, 0.19453658009801122, -0.10008771542481043, 0.0761802220
5035924, -0.3576203818599827, -0.109350268667331, -0.22707116400153105, 0.27
408709273244747, 0.2691326196127025, -0.011943072785749576, -0.4187424081959
703, 0.3366059649281856, 0.25321126090955515, -0.05837372712997879, -0.42172
810138930295, 0.287574916966417, 0.38096059740304544, 0.4983097900000707, 0.
45138238772850936, 0.34580546297509795, -0.40252832652595105, 0.317216472214
5582, 0.26269668401065094, 0.15061381349150627, -0.3370565906100078, 0.11000
388047252696, 0.30692775429576535, -0.011979632916260696, 0.1716284841239751
3, -0.12808767938662347, 0.1113142982664147, 0.07159744523085287, 0.37811473
67471953, 0.4638722850945386, -0.4384765458118387, -0.49639394840384277, -0.
313554078447128, -0.339946344881369, -0.13846587056056692, -0.09157487194127
611, 0.26879180542291903, -0.22230248088320714, -0.3305231915523701, -0.2180
9445474422406, -0.34275861105994254, 0.19804655556709416, 0.454882908028024,
0.22004140669285532, -0.11644487436429496, -0.1531933105427078, -0.006273197
857117774, 0.2777101541341257, 0.21739919180249756, 0.33323262169260315, -0.
4039382444271329, 0.045170652642498754, 0.20586307013678284, -0.050535882623
35185, 0.13413452066673237, -0.04441709683976591, -0.4664278755998238, 0.068
74286824467024, 0.25469671378824355, -0.15062443238263046, -0.13593240155270
525, 0.12390264883207325, -0.44481277364463734, -0.2113402733341586, 0.17687
35831854905, -0.07387214810686804, -0.17985596927209213, -0.4510511537220096
6, 0.23783631672344663, -0.2259882572014007, -0.27551815433431337, 0.2474036
1229336294, 0.2206532912941912, 0.38479778508873064, 0.22163069896013732, 0.
16008192465760052, 0.21864479399602454, 0.2225257093549735, -0.4239698420382
6094, 0.21573347184952996, 0.17737880733159483, -0.133098230222795, -0.19361
939501409742, -0.2964020315846505, -0.45115611260672905, -0.372857885566115
2, 0.014754665605435702, 0.19950608150417448, -0.024244574276777286, 0.35325
36337650253, 0.1445474586389257, 0.11334119742721038, 0.4966572487788411, 0.
25132244748098853, 0.3628501490468009, 0.24744458350653387, -0.4849236860228
069, 0.15904059937447657, 0.2593089675785111, -0.1435567622837094, 0.0855373
25688536, 0.19424370433625604, 0.216601406950914, 0.12016835374824619, -0.37
535106721981815, -0.03919998022144555, 0.059349950327841516, -0.399612055161
49956, 0.3423999054843173, 0.15134058081387813, 0.14300608629645084, -0.0900
8610845077025, 0.3863600707807173, 0.4235072371907139, 0.19039801607094697,
0.04615853048830265, -0.48127476397177527, 0.1274427374921383, 0.01777210985
599864, -0.3734619882955592, 0.41465468731648547, -0.09418087051534885, 0.37
14522776350969, -0.19403721838949095, -0.01635086360731397, 0.38397244958864
585, -0.4871637433745757, -0.016097389840506904, -0.03193330182662457, 0.232
92404646521514, 0.4694848482692836, -0.31905763855345737, -0.333237953744910
05, 0.19150624799971538, -0.27446608487001267, 0.023632403838000138, 0.01661
660086163863, 0.21487162991559494, 0.4585146721335934, -0.07014109621008235,
-0.1381332473310869, 0.4770655274784409, 0.05499873896944851, 0.211431173705
97664, 0.4374786226637165, 0.33942490012475623, 0.38861837669547294, -0.1399
6164167957692, 0.14927199896075571, -0.07827258936838954, 0.334830118275254
2, 0.22091071150785213, -0.4528007906307566, 0.331999138048893, 0.0820978011
2133092, -0.4355013696297588, -0.14117577703301254, 0.0489336834643036, 0.24
108648380584075, 0.26006184935592813, -0.1270100407391226, -0.32844044504976
766, 0.10001397294884706, -0.2601178329501902, -0.007075266104980549, -0.153
61572750821073, -0.30838993364361644, 0.12195482980158556, 0.314489858586642
6, -0.40853529290284674, 0.21815381035941173, 0.14869069009800084, -0.409899

51215746245, -0.36768909957450246, -0.19727090423866556, 0.3632006412233635, 0.18398047685614316, 0.16644244775685157, -0.4183967513021536, 0.10437933533885013, 0.03214915899308868, 0.20655691942637244, -0.3769421769557951, -0.3531419453333704, -0.39310148778502374, -0.17564651465032322, -0.38651389752352994, 0.09657849733181956, 0.12145291896243748, 0.11148644301933652, -0.2484411013625093, 0.09003763567262901, 0.23559161164370268, -0.014998788772455196, 0.3745401258149651, 0.3408689941557346, -0.20837120982778345, -0.02439545171964852, -0.16043197760524153, 0.3735473762278394, 0.45713363481959757, -0.15417075270581349, -0.0009345539999990882, 0.21769486817922812, 0.2083549191849262, 0.04707752232781126, 0.16740264947533123, 0.2897429667317363, 0.2858228768809542, 0.12505193942436132, 0.49134952544271093, -0.34656242272231064, 0.45912341801624046, 0.4034303223891972, 0.24167712157828514, 0.311793622945263, 0.28383660994929794, 0.02904008047993689, 0.19267956416379595, 0.318392579989489, -0.11210239643418307, 0.43710218077305374, 0.29772021665423076, -0.1901888486900244, 0.2843787383805605, -0.4591801284750213, -0.3309918173038987, -0.11552575496679207, 0.11156882655447198, -0.42677613538202785, 0.33577643526955714, -0.22842922886931372, -0.2756122659165644, -0.2184869450730187, -0.2718372094401468, 0.348460805038169, -0.1802096742303495, -0.28099104069552283, -0.4301830253448141, -0.2651953294448094, 0.07329828344822631, -0.22854019726192387, 0.13664839846952082, -0.2833503527237474, 0.06765115578338776, 0.46828250674322036, -0.01803515753797691, 0.27156959965692573, -0.4026630933706419, 0.21608005663132157, 0.035262470902371, -0.0462837279509134, 0.15882071074053317, 0.23327798641801445, -0.07819081647582649, -0.1542257546000304, 0.279968382807232, 0.3650853210181598, -0.4545502824878481, 0.20207364093344782, 0.30285597304432255, -0.2068749363351673, 0.38407821208644044, -0.18020027084048806, 0.2744029128761384, -0.1863046552276162, 0.30620257665021533, 0.10506419168815129, 0.135694848409609, 0.4738417106023066, -0.16127513002463445, -0.12999537005919992, -0.04427906464017006, -0.43891199773686707, 0.4066492390811537, -0.06657817986254089, 0.40601094360263046, 0.027726828615830423, 0.36527955427830994, -0.32890242207867404, -0.13556862349352317, -0.31058163140875217, 0.4740671190129312, -0.3342243917850467, -0.20221824879519268, -0.17259023292299625, 0.25004782641538725, -0.275761747501148, -0.05358401822321346, 0.26068637089509716, -0.2406044417493658, -0.0904401497341576, 0.28011822802295494, -0.1499336571854687, -0.22023637293965181, -0.21910122923005326, 0.39186591475119736, -0.23741780312038596, 0.04328206553695535, -0.10079699569706446, 0.008325348234323626, 0.37652774577697046, -0.4034910486935547, 0.08885626991888618, 0.36155070761045405, -0.26718525846257557, 0.49343032089319516, 0.4300576594079438, 0.2826288739381434, 0.32388705896282555, 0.28454471972059814, -0.39531934270509794, 0.20223992228659615, -0.33484728272423436, -0.13405309487576345, -0.42950382171125834, -0.02738596577943142, -0.2011032976488718, 0.029102746127841472, -0.48961980532395744, 0.30401606096029143, -0.07970808923860662, -0.10036245503324492, 0.349218558236294, 0.13698393525590535, 0.4001495235255147, -0.473990633958122, -0.4954054475461106, 0.020499174525744457, -0.4066188956618313, -0.352772074265271, -0.071797122510034, 0.3223816921543534, -0.4229705554029436, 0.2701353064293197, -0.06962263890446607, -0.0996645922156304, 0.09113859949480785, -0.05749942128246921, -0.3557275710271436, -0.36397458630086654, -0.23787956578580038, -0.17534930242061142, -0.2743884868933253, -0.020578956003559745, 0.3173146210264528, 0.39204445224155016, 0.3721028557718141, 0.17638000864064085, 0.11503901614781653, -0.02222072488897131, -0.3742440826548812, -0.1286692131216064, 0.1547675988098436, 0.3752717436908921, 0.31046525525597524, 0.27726688710919933, 0.306434926473914, 0.4075684205950695, 0.42037087772642745, 0.20106875129785606, -0.4635218190923799, 0.3682979314861913, -0.3520404808428088, -0.4046546084514381, 0.3867127807519902, -0.20548153811884073, 0.15571772510664517, 0.01605530401658284, 0.14452824679451348, 0.40535706267342875, 0.013109793261834701, -0.03962448241124372, 0.012213169964417236, -

0.23405459909182214, 0.19196868295957736, 0.47139156268328597, -0.3843933615
8416193, 0.25375527742958626, 0.4170795894636806, -0.036537815934459594, 0.4
0740109656187007, 0.0751031417671314, 0.05419676550607888, 0.454326798097650
64, -0.45270080284327974, 0.05076806941281853, -0.07115247507188627, 0.25254
19831787269, -0.3166651744676664, 0.01985824901136113, -0.33299778918488565,
0.440485850342905, 0.34995956998131095, -0.33947377348334584, -0.48621254866
09443, 0.22869493058115475, -0.07353753789667894, -0.14548098154835354, 0.38
11065677094976, 0.061395650025173, -0.1783441707528025, -0.4117256766127454,
-0.2557114390018942, -0.44065057915255923, 0.47084451238013825, 0.2296262080
760091, 0.20194450626798643, 0.4666091597555909, -0.06004067804798263, -0.28
95954654474254, 0.3914579019782588, 0.12266649264584184, -0.0742694790291720
3, -0.0161038983050793, 0.2370571441067485, 0.45386382131777847, -0.40692912
107202495, 0.21095807881907802, -0.44608250634741975, -0.09028745099215307,
-0.4064366233898252, 0.1882396635303032, 0.060840813926123394, 0.28850803820
426274, -0.49336818443241337, 0.26087355725558903, 0.36433703085829405, 0.28
200246523804917, -0.014523215301399328, 0.3320394078201415, -0.3141217816839
459, 0.026984292331661663, 0.4040895330895927, -0.0827537244645643, -0.10232
942217999386, 0.2177331681662028, -0.33398814878527816, 0.4226369779852259,
-0.44342198102068286, -0.15318277350741183, -0.4560844593070461, -0.19393502
950558295, -0.3711377468181175, -0.11340138861903781, 0.4348815011109747, 0.
4417937493151233, -0.43149923233876075, 0.19775539016239363, -0.238267257224
49005, -0.46905084759804394, 0.2683442711073585, 0.4705103928176818, 0.03350
839032827402, 0.04010946632892787, 0.10760776965123087, 0.03962622614510935
6, -0.18483521791679514, 0.03115576722785174, -0.1253866620564439, -0.169641
61402683664, 0.07160408956795417, -0.3063008584635606, 0.16136414134268806,
0.4975162051057098, 0.47360804989667904, 0.26108079568740894, 0.286210578874
14925, 0.4969492396487999, 0.13646947437460322, 0.1687295278837242, 0.149088
74831239827, 0.08249862671543295, 0.498021565797679, -0.33714018001178503, -
0.3316941648259658, 0.29599303166895163, -0.26307573604320655, -0.0120398610
37640875, -0.41776382726730943, 0.35044233704611183, -0.15896513446058647,
0.49051684484737323, 0.3214426378789541, 0.025686313020127804, 0.15103242058
456479, -0.38865705593606614, -0.19960667291663015, 0.21797005002533187, 0.0
4252098392531434, -0.4080260685038757, 0.17622666467747972, 0.02020083637196
135, 0.34832529197918616, 0.4844913699321025, -0.4608001363462979, 0.3746176
0887724427, -0.30717387776168437, -0.1148807394043484, -0.42773096409655653,
0.31661869309730484, -0.19010462990211074, 0.4896226207295009, -0.2400632103
1228373, 0.43291332125185245, -0.06918460573150997, -0.20617487963254844, 0.
3256749216450483, -0.21713725959478092, -0.06974475036077188, -0.38202692854
9984, 0.41567266833430994, -0.05259081954976219, -0.3593271651327138, 0.3773
5673239564604, 0.4740703927812828, 0.21631372788782932, 0.40909796229039974,
-0.29946034313534786, 0.29642337879483804, -0.48647547379633227, -0.08016550
207011108, 0.245670267142987, -0.12259252336034188, -0.35032564258837395, -
0.3128119883687921, -0.15942055213944561, 0.08994089080143086, -0.1510378979
285465, -0.3708453203480805, -0.3053760236792752, 0.0024891475523208806, 0.2
597717750705164, 0.11716378090900814, -0.4339750892626526, -0.23291559259706
65, 0.14862863476904953, -0.08270374112213241, 0.05052351893451135, -0.13682
2485329678, 0.31077290280370795, -0.04092878156978719, -0.44374705364944034,
0.10103882717029633, -0.3862138835127913, -0.09926254217424735, -0.499188843
45224024, 0.08038607830390732, -0.4349326811795662, -0.09145393985143202, -
0.08096733421314939, 0.0924431705399622, 0.05990622626195097, -0.09816295655
622065, 0.08898695376541066, 0.42247301414044636, -0.27451434468080294, -0.0
2398277422616346, -0.3777929724871205, -0.13813331252355443, 0.1716212593524
7052, 0.06684151148704076, -0.28460633968051074, 0.017088906640729706, 0.155
77422340988023, -0.18389643412457113, 0.2125222163781162, -0.051898296607363
78, -0.3329091241333141, 0.27984929448443663, -0.07777554818615051, -0.19658
301581240134, 0.004047142665075665, -0.46229788762892576, 0.2826917943395045

3, 0.29730619147999726, -0.4709634222408101, -0.4226898137593246, -0.3617351
557648786, -0.016491153824418125, 0.02485805740638425, 0.006768313156715333,
0.07719585036762355, -0.16293706946738262, -0.3105076225426807, 0.3660202274
6250074, 0.467117407438558, -0.42905388549472534, -0.14882314789932516, -0.0
4495835659399394, -0.21733443245374318, 0.04658415584347608, 0.2453560461225
982, 0.3769155244099067, 0.4400684412555462, 0.2218424819635657, -0.12053863
577426882, -0.4324453114794645, -0.28864465454782606, 0.4503660981731388, 0.
02323504120327602, 0.3408246710279249, 0.06563110755555013, -0.4311765316990
863, 0.11274820811845021, 0.36994252571766995, 0.43427396859052714, 0.339350
33534461634, -0.1500435034702673, -0.2493348897597253, -0.1801162840446987,
0.23494399120323872, 0.14516930354445579, -0.22507355508933535, 0.3627503699
7248766, -0.35788109367036525, -0.4028161022011887, 0.03966348445175427, -0.
22996557391598194, 0.19264497794500635, 0.362373347175743, -0.08916578271360
975, 0.19358924834475966, -0.49255654780254843, -0.39253574497695587, -0.238
9067381793697, -0.10409751051654748, 0.46758609042121746, -0.424502869147954
43, 0.24809271225320995, -0.17935065009789997, -0.014740642142665461, -0.355
83632017158295, -0.2110798353156843, 0.056519068355929125, -0.44538935441549
676, -0.36146008792583995, -0.0790638524646925, 0.03163929318300818, -0.2436
2560946134337, 0.05022389279436412, -0.28935376878423014, -0.073731873743159
04, 0.0215860434515881, -0.09145958003718446, 0.14967590021651456, -0.081089
13239858329, -0.3723085362523292, -0.09271539137475304, -0.03668884988316134
5, 0.4117434328827676, -0.30800403788490993, -0.34523349202136633, 0.3289616
497640687, -0.2934746056229164, -0.4841851928765085, 0.39209535721808586, 0.
20608085366331408, 0.023049684121085656, 0.2502343673324522, -0.064985153930
12805, 0.3631523176443534, 0.13017140356651802, 0.35062838934441865, -0.0762
2036486144868, -0.4599136815493703, 0.36867879867228526, -0.4820189812115146
5, -0.4031178715121816, 0.25897998414845647, 0.4681929580070767, -0.44142021
16248621, -0.11815196452421184, -0.10518676989550091, -0.07361138782226428,
0.003826910449118537, -0.09704711804226152, -0.26514270737027923, 0.03035898
191362585, -0.1224766660775487, 0.2171448714265244, 0.10116272419463535, 0.2
992793641459529, 0.12794753128957537, -0.008567471677206062, -0.427169865671
0261, -0.4779138850974677, 0.2642323889549275, -0.2905174296998745, -0.11871
072393921334, 0.2992215273904394, 0.40060249659789005, 0.1416406493732456, -
0.1564685012436674, -0.3351061815526115, -0.31045039849931, 0.33876431797033
335, -0.10045647676992442, -0.21383089969528923, -0.460163796027684, -0.1517
6369391932454, 0.47290499679940745, -0.14868516437362367, 0.308622418147921
7, 0.03906457045419287, -0.08508249017317226, -0.29844102017041463, -0.49746
27053028221, 0.28198798397513447, 0.25030645897167736, -0.01091090292060292
7, 0.14369834685588956, -0.4856858153684216, 0.09130429522752348, 0.15749289
599267213, -0.012194853138575601, 0.21915798070457237, -0.45621070707174205,
0.11589182697650469, -0.48865900282341845, 0.37475971503295424, 0.2798670382
596351, 0.014513887645111412, 0.21986541498942858, -0.290567420104454, 0.188
4564540825493, -0.23595970146863188, 0.35444421843848806, 0.0326097960010186
54, 0.007620064786187486, -0.07930056606615543, -0.2466718175698458, 0.02588
8788013680886, -0.3175111986567223, 0.0730156943927237, -0.3105200009850497,
0.3179178201583053, 0.0843694392422919, 0.20002855986684132, -0.157206725463
30883, 0.021994163874108574, -0.12250123460776685, -0.47139830220552315, -0.
1715143730220955, 0.12270355856835924, -0.14082205380978974, -0.259422206436
42977, -0.13530582371246436, -0.12515198465598631, -0.4978646212817436, 0.24
030483270455794, 0.21993717041249716, 0.2580744625769653, -0.216098988281735
46, -0.4771280133613043, -0.04251865945032973, -0.2873334019687934, -0.07221
454410453187, -0.38712581726332196, 0.07793700771241463, 0.4389503908252199,
-0.2337089015832472, 0.13250675888374663, 0.1701029484937252, -0.45332724205
642416, -0.4743008238822817, -0.4129715382326393, 0.23830378602543056, 0.062
71140871181402, 0.11506229654060751, -0.1638906511382653, -0.089792090916923
2]

```

1000-element Vector{Float64}:
 0.41919142411294885
-0.04477120169153992
-0.15987063495003362
-0.3165390583665356
-0.04345183269168251
 0.4600428661267356
-0.4814808404373695
 0.05265958609672772
-0.019033198532207418
 0.11752715148103854
  ⋮
 0.1701029484937252
-0.45332724205642416
-0.4743008238822817
-0.4129715382326393
 0.23830378602543056
 0.06271140871181402
 0.11506229654060751
-0.1638906511382653
-0.0897920909169232

```

What does this `MethodError` mean? Why does this code produce one? Fix the error and solve the problem.

The original code produced a `MethodError` because it attempted to subtract a scalar from a vector using `vect - m`. In Julia, this operation is not defined without broadcasting, since Julia does not automatically apply the subtraction element by element. To fix this error, I changed the line to `vect .- m`, which broadcasts the subtraction across all elements of the vector. I also initialized the running sum as 0.0 so that the calculation works smoothly with floating-point numbers. After making these changes, the function successfully returned a demeaned vector, and checking its mean confirmed that it was very close to zero, which shows that the problem was solved correctly.

Problem 2 (18 points)

Cheap Plastic Products, Inc. is operating a plant that produces $100\text{m}^3/\text{day}$ of wastewater that is discharged into Pristine Brook. The wastewater contains $1\text{kg}/\text{m}^3$ of YUK, a toxic substance. The US Environmental Protection Agency has imposed an effluent standard on the plant prohibiting discharge of more than $20\text{kg}/\text{day}$ of YUK into Pristine Brook.

Cheap Plastic Products has analyzed two methods for reducing its discharges of YUK. Method 1 is land disposal, which costs $X_1^2/20$ dollars per day, where X_1 is the amount of wastewater disposed of on the land (m^3/day). With this method, 20% of the YUK applied to the land will eventually drain into the stream (*i.e.*, 80% of the YUK is removed by the soil).

Method 2 is a chemical treatment procedure which costs $1.50 \text{ per } \text{m}^3 \text{ of wastewater treated}$. The chemical treatment has an efficiency of $e = 1 - 0.005X_2$, where X_2 is the quantity of wastewater (m^3/day) treated. For example, if $X_2 = 50 \text{ m}^3/\text{day}$, then $e = 1 - 0.005(50) = 0.75$, so that 75% of the YUK is removed.

Cheap Plastic Products is wondering how to allocate their wastewater between these three disposal and treatment methods (land disposal, chemical treatment, and direct disposal) to meet the effluent standard while keeping costs manageable.

The flow of wastewater through this treatment system is shown in [Figure 1](#). Modify the edge labels (by editing the `edge_labels` dictionary in the code producing [Figure 1](#)) to show how the wastewater allocations result in the final YUK discharge into Pristine Brook. For the `edge_label` dictionary, the tuple (i, j) corresponds to the arrow going from node i to node j . The syntax for any entry is `(i, j) => "label text"`, and the label text can include mathematical notation if the string is prefaced with an `L`, as in `L"x_1"` will produce x_1 .

```
In [ ]: A = [0 1 1 1;
           0 0 0 1;
           0 0 0 1;
           0 0 0 0]

names = ["Plant", "Land Treatment", "Chem Treatment", "Pristine Brook"]
# modify this dictionary to add labels
# old code: edge_labels = Dict{(1, 2) => "", (1,3) => "", (1, 4) => "", (2,
edge_labels = Dict{(1, 2) => L"x_1", (1,3) => L"x_2", (1, 4) => L"100-x_1-x_
shapes=[:hexagon, :rect, :rect, :hexagon]
shapes=[:hexagon, :rect, :rect, :hexagon]
xpos = [0, -1.5, -0.25, 1]
ypos = [1, 0, 0, -1]

p = graphplot(A, names=names, edgelabel=edge_labels, markersize=0.15, markers
display(p)
```

I added labels to the `edge_labels` to show each flow with units of kg/day of YUK as the concentration of YUK is 1 kg/m^3 .

Problem 2.1

Formulate a mathematical model for the treatment cost and the amount of YUK that will be discharged into Pristine Brook based on the wastewater allocations. This is best done with some equations and supporting text explaining the derivation. Make sure you include, as additional equations in the model, any needed constraints on relevant values. You can find some basics on writing mathematical equations using the LaTeX typesetting syntax [here](#), and a

cheatsheet with LaTeX commands can be found on the course website's [Resources page](#).

Treatment cost = land treatment cost + chem treatment cost + direct disposal cost

$$\text{Treatment cost} = X_1^2/20 + 1.5X_2 + 0$$

$$\text{Treatment cost} = X_1^2/20 + 1.5X_2 \text{ (dollars/day)}$$

The total treatment cost is the sum of all three methods of treatment. As specified above, land treatment is $X_1^2/20$ dollars/day while chemical treatment is $1.5X_2$ dollars/day. Direct disposal is assumed free because there is no information on cost.

YUK discharge = land treatment discharge + chem treatment discharge + direct discharge

$$\text{YUK discharge} = 0.2X_1 + (1 - e)X_2 + (100 - X_1 - X_2)$$

$$\text{YUK discharge} = 0.2X_1 + (1 - (1 - 0.005X_2))X_2 + (100 - X_1 - X_2)$$

$$\text{YUK discharge} = 0.2X_1 + (0.005X_2)X_2 + (100 - X_1 - X_2)$$

$$\text{YUK discharge} = 0.2X_1 + 0.005X_2^2 + (100 - X_1 - X_2)$$

where $X_1 + X_2 \leq 100$ and YUK discharge < 20 kg/day.

The total YUK discharge is the sum of all three discharge sources. Land treatment discharges 20% of the original YUK ($0.2X_1$). Chemical treatment discharges $(1 - e)\%$ of the original YUK $((1 - e)X_2) = 0.005X_2^2$. Direct disposal discharges the untreated portion $(100 - X_1 - X_2)$. Since influent is 1 kg/m³, discharge (m³/day) is equal to discharge (kg/day).

Problem 2.2

Implement your systems model as a Julia function which computes the resulting YUK discharge and cost for a particular treatment plan. You can return multiple values from a function with a [tuple](#), as in:

```
In [ ]: function multiple_return_values(x, y)
          return (x+y, x*y)
        end

a, b = multiple_return_values(2, 5)
@show a;
@show b;
```

To evaluate the function over vectors of inputs, you can *broadcast* the function by adding a decimal `.` before the function arguments and accessing the resulting values by writing a *comprehension* to loop over the individual outputs in the vector:

```
In [ ]: x = [1, 2, 3, 4, 5]
        y = [6, 7, 8, 9, 10]

        output = multiple_return_values.(x, y)
        a = [out[1] for out in output]
        b = [out[2] for out in output]
        @show a;
        @show b;
```

```
In [ ]: ##### Problem 2.2

        # Function returns (cost $/day, YUK discharge kg/day) given X1, X2
        function evaluate_plan(X1, X2)
            cost = (X1^2)/20 + 1.5*X2
            yuk = 0.2*X1 + 0.005*X2^2 + (100 - X1 - X2)
            return (cost, yuk)
        end

        # Test three example allocations
        @show evaluate_plan(50, 50) # split evenly
        @show evaluate_plan(10, 90) # mostly chemical
        @show evaluate_plan(0, 0)  # all direct
```

For Problem 2.2, I wrote a Julia function called `evaluate_plan` that takes the flows to land treatment (X_1) and chemical treatment (X_2) as inputs and returns a tuple containing both the treatment cost (in dollars per day) and the YUK discharge (in kilograms per day). The cost is computed using the formula $(X_1^2)/20 + 1.5X_2$, while the discharge is computed as $0.2X_1 + 0.005X_2^2 + (100 - X_1 - X_2)$, which represents the combined contributions from land treatment, chemical treatment, and direct discharge. I tested the function on three example plans: one with an even 50/50 split between land and chemical treatment, one with mostly chemical treatment (10/90), and one with all direct discharge (0/0). The results confirmed that the function correctly calculates both the cost and the YUK discharge for different treatment allocations.

Make sure you comment your code appropriately to make it clear what is going on and why.

Problem 2.3

Use your function to experiment with 1,000 different combinations of wastewater discharge and treatment. You can do this with either a grid search or by sampling from a [Dirichlet distribution](#) (a `Dirichlet(1, n)` distribution will generate

uniformly-weighted n -dimensional vectors whose components add up to 1; see the `Distributions.jl` [documentation](#) for how to sample from probability distributions in Julia). Plot the results of these experiments. Do any satisfy the YUK effluent standard (plot this as well as a dashed red line). What was the cost of solutions satisfying the standard? What can you say about the tradeoff between treatment cost and YUK discharge? You don't have to find an "optimal" solution to this problem, but what do you think would be needed to find a better solution?

Problem 2.4

Find the strategies which minimize cost and YUK discharge (these will be different strategies) analytically and find the values of the objective metrics. Plot these values in the plot that you created for Problem 2.3. How do their values compare to the spread of values that you found in that problem? Would you select either of them (explain why or why not)?

References

List any external references consulted, including classmates.

```
In [ ]: ##### Problem 2.3

using Distributions, Random, Plots

# --- Model: given flows to land (x1), chemical (x2), direct (x3)
# Returns (discharge_kg_per_day, cost_dollars_per_day, is_feasible)
function model(x1::Real, x2::Real, x3::Real)
    z = 0.2*x1 + 0.005*x2^2 + x3          # YUK to brook (kg/day)
    c = (x1^2)/20 + 1.5*x2               # daily cost ($/day)
    return (z, c, z <= 20)
end

# --- Sampling setup
seed = rand(1:100)                      # random seed (recorded below)
Random.seed!(seed)

Q = 100.0                               # total flow (m^3/day)
n = 1000                                # number of combinations

# Dirichlet over 3 parts; each column sums to 1 -> scale by Q
d = Dirichlet(ones(3))
X = Q .* rand(d, n)                    # 3xn matrix

# Evaluate all plans
results = [model(X[1,j], X[2,j], X[3,j]) for j in 1:n]

# Unpack tuples
z = first.(results)                     # discharge (kg/day)
c = getindex.(results, 2)               # cost ($/day)
```

```

feasibility = getindex.(results, 3) # Bool

# Feasible set indices
feasible = findall(feasibility)

# Report cheapest feasible and lowest-discharge feasible (if any)
if !isempty(feasible)
    min_idxc = feasible[argmin(c[feasible])] # cheapest feasible
    println("Results for Lowest Cost Plan:")
    println("z = $(round(z[min_idxc], digits=3)) kg/day")
    println("c = $(round(c[min_idxc], digits=2)) dollar/day")

    min_idxz = feasible[argmin(z[feasible])] # lowest discharge feasible
    println("Results for Lowest Discharge Plan:")
    println("z = $(round(z[min_idxz], digits=3)) kg/day")
    println("c = $(round(c[min_idxz], digits=2)) dollar/day")
else
    @warn "No feasible plans ( $z \leq 20$  kg/day) found in this sample."
end

println("random seed = $seed")

# --- Plot: cost vs discharge
plt = scatter(
    z, c;
    xlabel = "YUK discharge to brook (kg/day)",
    ylabel = "Daily treatment cost (dollars/day)",
    title = "Daily Treatment Cost vs. YUK Discharge To Brook",
    label = "Sampled plans",
    legend = :topright,
    ms = 3
)

# vertical effluent limit at 20 kg/day
vline!(plt, [20.0]; lc = :red, ls = :dash, label = "Discharge Limit = 20 kg/day")

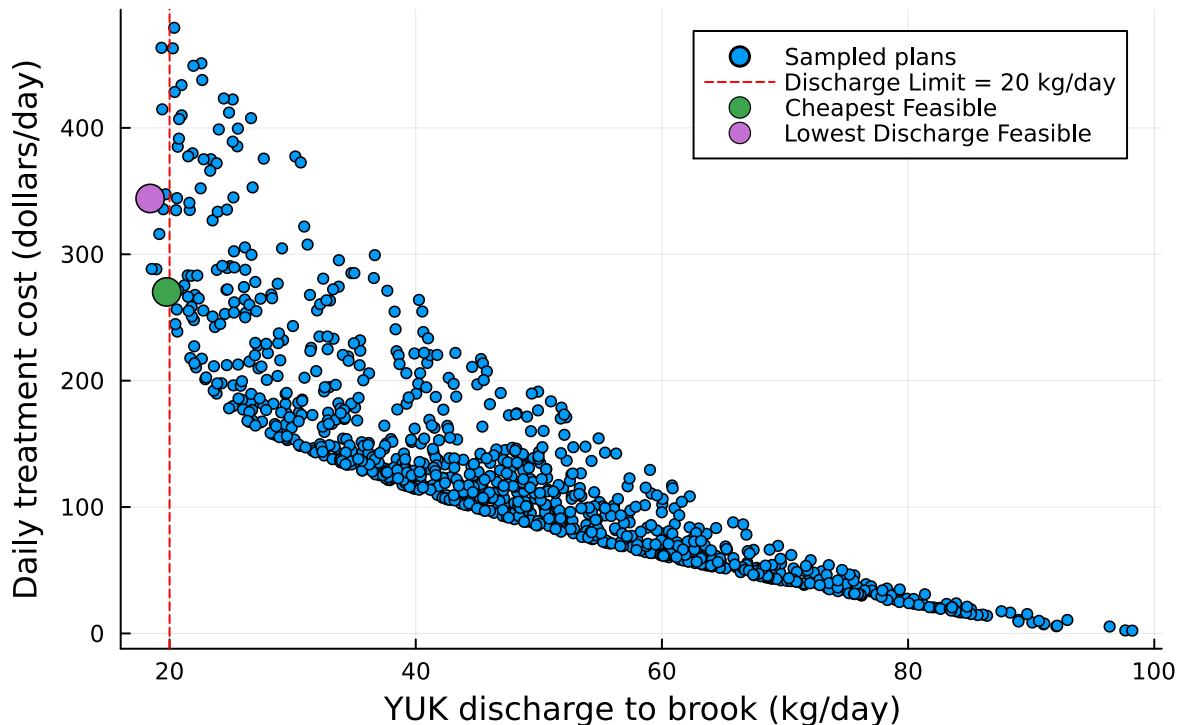
# highlight the two special feasible plans
if !isempty(feasible)
    scatter!(plt, [z[min_idxc]], [c[min_idxc]]; ms=8, label="Cheapest Feasible")
    scatter!(plt, [z[min_idxz]], [c[min_idxz]]; ms=8, label="Lowest Discharge Feasible")
end

display(plt)

```

Results for Lowest Cost Plan:
z = 19.768 kg/day
c = 270.19 dollar/day
Results for Lowest Discharge Plan:
z = 18.422 kg/day
c = 344.15 dollar/day
random seed = 39

Daily Treatment Cost vs. YUK Discharge To Brook



For Problem 2.3, I created a Julia function called `model` that takes in the flows to land treatment, chemical treatment (x_2), and direct discharge (x_3) and returns the YUK discharge, treatment cost, and whether the plan is feasible (i.e., discharge ≤ 20 kg/day). Using the Dirichlet distribution, I generated 1,000 random allocations of the 100 m³/day wastewater among the three options, ensuring that the flows always sum to the total. For each plan, I calculated the discharge and cost, then identified the subset of plans that met the effluent standard. I reported both the cheapest feasible plan (minimum cost while satisfying the limit) and the lowest-discharge feasible plan. Finally, I plotted all sampled plans as a scatter plot of cost versus discharge, added a vertical dashed line at the 20 kg/day standard, and highlighted the two special feasible plans on the graph. This experiment shows the tradeoff between cost and discharge, with low-cost plans generally producing higher discharges, and feasible plans clustering at higher costs.

```
In [9]: ##### Problem 2.4
using Distributions, Random, Plots

seed = rand(1:100)
Random.seed!(seed)

Q = 100.0
n = 1000

d = Dirichlet(ones(3))
X = Q .* rand(d, n)
```



```

model(x1, x2, x3) = begin
    discharge = 0.2*x1 + 0.005*x2^2 + x3
    cost = (x1^2)/20 + 1.5*x2
    feasible = discharge <= 20
    return (discharge, cost, feasible)
end

results = [model(X[1,j], X[2,j], X[3,j]) for j in 1:n]

z = first.(results)
c = getindex.(results, 2)
feasibility = getindex.(results, 3)

feasible = findall(feasibility)
println("Random seed = $seed")
println("Number of feasible plans = ", length(feasible))

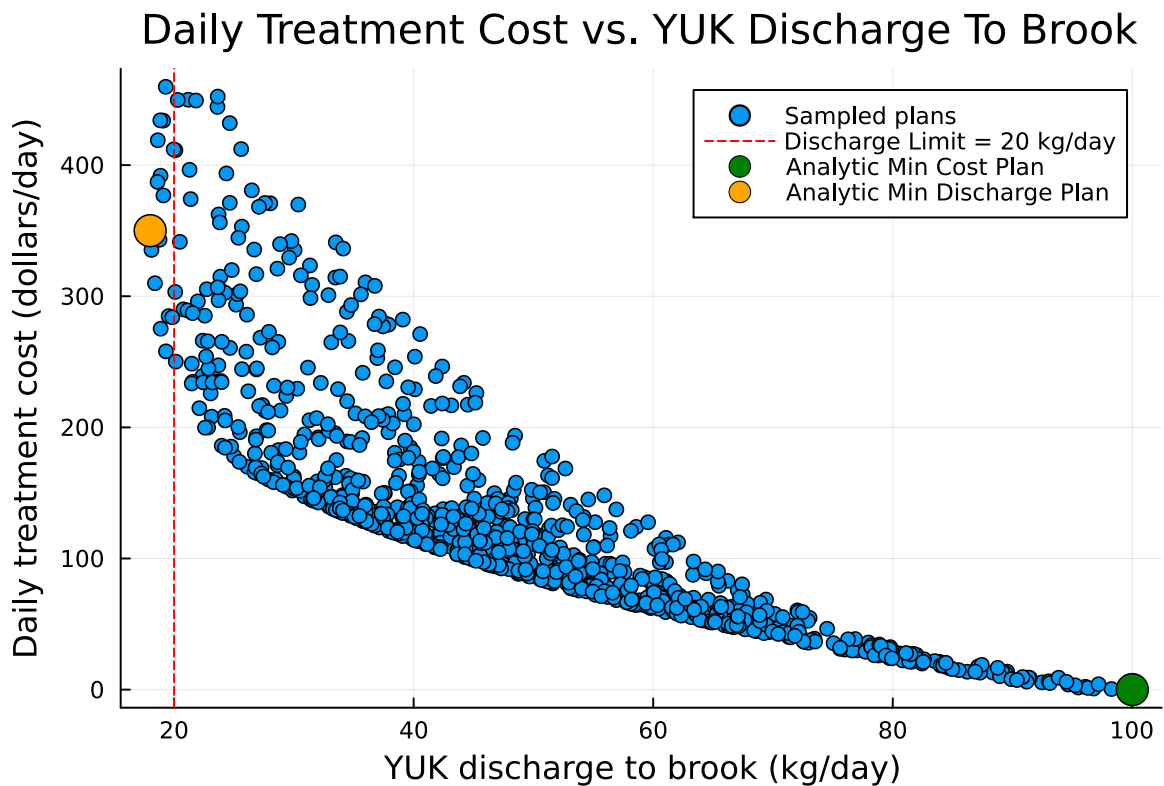
plt = scatter(z, c; xlabel = "YUK discharge to brook (kg/day)",
    ylabel = "Daily treatment cost (dollars/day)",
    title = "Daily Treatment Cost vs. YUK Discharge To Brook",
    label = "Sampled plans", legend = :topright)

vline!(plt, [20.0]; lc = :red, ls = :dash, label = "Discharge Limit = 20 kg/day")

scatter!(plt, [100.0], [0.0]; ms=9, c=:green, label="Analytic Min Cost Plan")
scatter!(plt, [18.0], [350.0]; ms=9, c=:orange, label="Analytic Min Discharge Plan")

display(plt)

```



Random seed = 51
 Number of feasible plans = 16

For Problem 2.4, I analytically identified the two extreme strategies and overlaid them on a scatter plot of 1,000 randomly sampled treatment plans. The code samples feasible allocations with a Dirichlet distribution (flows sum to 100 m³/day), computes each plan's YUK discharge and daily cost with model, and marks plans that satisfy the effluent limit (≤ 20 kg/day). I then added the analytic minimum-cost point, which sends all flow directly to the brook ($x_1 = 0, x_2 = 0, x_3 = 100$) ($x_1 = 0, x_2 = 0, x_3 = 100$) and therefore has cost = 0/day and discharge = 100 kg/day (not feasible), and the analytic minimum discharge = 0 with discharge = 18 kg/day and cost = 350/day (feasible). The plot makes the tradeoff clear: achieving very low discharge requires substantially higher cost. Comparing these extremes to the random cloud, the min-cost plan is unacceptable because it violates the standard, while the min-discharge plan is safe but expensive. A more practical choice would be the cheapest feasible plan near the 20 kg/day boundary, which could be found more reliably with a constrained optimization approach rather than random sampling.